

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

COMPUTER NETWORKS

Bài tập lớn 1

c

Instructor: Bùi Xuân Giang

Class ID: CO3094 (A01)

Students: Nguyễn Chí Thanh 2313078

Lê Đức Tài 2312995

Nguyễn Việt Hoàng 2311066

Mai Xuân Nhựt 2312549

HỒ CHÍ MINH, 2026

Mục lục

1 Tổng quan kiến trúc hệ thống

Hệ thống được xây dựng theo mô hình đa tiến trình gồm ba thành phần chính:

- **Backend Server** (daemon/backend.py): Xử lý HTTP request, xác thực, phục vụ file tĩnh và API RESTful.
- **ChatApp / AsynapRous** (apps/chatapp.py, daemon/asynaprous.py): Ứng dụng chat kết hợp Client-Server và Peer-to-Peer.
- **Tracker** (apps/trackerapp.py): Quản lý đăng ký và khám phá peer.

Toàn bộ hệ thống sử dụng **Python standard library** (socket, select, threading) mà không dùng bất kỳ web framework hay asyncio bên ngoài nào. Thay vào đó, nhóm tự xây dựng một **Event Loop** dựa trên select() hệ thống.

2 Cơ chế Non-blocking (Phần 2.1)

2.1 Hai chế độ xử lý đồng thời

Hệ thống hỗ trợ hai cơ chế được lựa chọn qua tham số -mode:

Cơ chế	Mô tả	Lệnh
Threading	Mỗi kết nối tạo một daemon thread riêng	-mode threading
Callback (mặc định)	Custom select() Event Loop, 1 thread phục vụ nhiều kết nối	-mode callback

2.2 Chế độ Callback — Custom select() Event Loop

Đây là đóng góp trọng tâm của bài tập: tự xây dựng Event Loop sử dụng lời gọi hệ thống select.select() thay vì dùng asyncio.

Kiến trúc Event Loop (daemon/eventloop.py)

Listing 1: EventLoop — vòng lặp sự kiện tự xây

```

1 class EventLoop:
2     """
3     Vòng lặp sự kiện (singleton) dựa trên select.select().
4     1 thread duy nhất phục vụ nhiều kết nối đồng thời.
5     """
6     def _run_once(self):

```

```

7      # Buoc 1: Chay pending callbacks (TaskQueue)
8      self._task_queue.drain()
9      # Buoc 2: Hoi OS socket nao san sang (non-blocking)
10     r_list = self._registry.get_read_sockets()
11     readable, writable, _ = select.select(
12         r_list, [], [], self._select_timeout
13     )
14     # Buoc 3: Goi callback tuong ung
15     for sock in readable:
16         cb = self._registry.get_read_callback(sock)
17         if cb:
18             cb(sock)
19
20     def run_forever(self):
21         self._running = True
22         while self._running:
23             self._run_once()

```

Nguyên lý 1 iteration (1 tick):

1. Drain TaskQueue — chạy các callback đã lên lịch.
2. Gọi `select.select()` — hỏi OS danh sách socket sẵn sàng I/O.
3. Gọi callback đã đăng ký cho từng socket sẵn sàng.
4. Lặp lại — không bao giờ dừng cho đến khi `stop()`.

CallbackRegistry — Danh bạ socket → callback

```

1 class CallbackRegistry:
2     """Anh xa: socket -> callback function."""
3     def register_read(self, sock, callback):
4         self._read_callbacks[sock] = callback
5     def unregister(self, sock):
6         self._read_callbacks.pop(sock, None)

```

SelectHTTPServer — HTTP Server tích hợp EventLoop

Listing 2: Luong xu ly ket noi moi

```

1 class SelectHTTPServer:
2     def start(self):

```

```

3         self._server_sock = socket.socket(AF_INET, SOCK_STREAM)
4         self._server_sock.setblocking(False)    # Non-blocking!
5         self._server_sock.bind((self.ip, self.port))
6         self._server_sock.listen(128)
7         self.loop.register_read(self._server_sock, self.
            _on_new_connection)
8
9     def _on_new_connection(self, server_sock):
10         conn, addr = server_sock.accept()
11         conn.setblocking(False)
12         self._buffers[conn] = ConnectionBuffer(conn, addr)
13         self.loop.register_read(conn, self._on_data)
14
15     def _on_data(self, conn):
16         data = conn.recv(4096)    # Non-blocking
17         buf = self._buffers[conn]
18         buf.feed(data)
19         if buf.done:               # Đã nhận đủ 1 HTTP request
20             self._handle_request(conn, buf)
21
22     def _handle_request(self, conn, buf):
23         adapter = HttpAdapter(self.ip, self.port, None, buf.addr, self
            .routes)
24         response = adapter.process_request(buf.get_request(), buf.addr
            )
25         conn.sendall(response)
26         self._close_conn(conn)

```

Phân tầng trách nhiệm:

- **EventLoop**: Quản lý vòng lặp `select()`, phân phối sự kiện.
- **SelectHTTPServer**: Quản lý kết nối TCP (`accept`, `recv`, `send`).
- **ConnectionBuffer**: Gom dữ liệu từng mảnh đến khi đủ 1 HTTP request.
- **HttpAdapter.process_request()**: Xử lý HTTP (`parse`, `auth`, `routing`).

Chế độ Threading

```

1  # daemon/backend.py --- threading mode
2  if mode_async == "threading":
3      client_thread = threading.Thread(

```

```

4         target=handle_client,
5         args=(ip, port, conn, addr, routes)
6     )
7     client_thread.daemon = True
8     client_thread.start()

```

Mỗi kết nối được giao cho một daemon thread xử lý độc lập. Phù hợp khi số kết nối đồng thời thấp, mỗi request cần nhiều CPU.

2.3 Khởi động server

```

1  # Callback mode (mac dinh) --- port 9020
2  python start_backend.py --server-port 9020 --mode callback
3
4  # Threading mode --- port 9010
5  python start_backend.py --server-port 9010 --mode threading

```

3 Xác thực HTTP (Phần 2.2)

3.1 Luồng xác thực trong HttpAdapter.process_request()

Toàn bộ logic HTTP (parse + auth + routing) được tập trung trong method `process_request()`, được gọi bởi cả 3 mode:

Listing 3: `process_request()` — loi HTTP đa dụng chung

```

1  def process_request(self, raw_msg, addr):
2      req.prepare(raw_msg, self.routes)    # Parse HTTP
3
4      # Bước 1: Kiểm tra Cookie session_id (RFC 6265)
5      if req.cookies and 'session_id' in req.cookies:
6          sid = req.cookies['session_id']
7          if sid in ACTIVE_SESSIONS:
8              is_authenticated = True
9              current_user = ACTIVE_SESSIONS[sid]
10         else:
11             user = validate_session(sid)
12             if user:
13                 is_authenticated = True
14                 add_session(sid, user)
15
16         # Bước 2: HTTP Basic Auth (RFC 7235)

```

```

17     if not is_authenticated and req.auth:
18         decoded = base64.b64decode(auth_parts[1]).decode()
19         username, password = decoded.split(':', 1)
20         if VALID_USERS.get(username) == password:
21             new_session = create_session(username)
22             add_session(new_session, username)
23             new_cookie_to_set = f"session_id={new_session}; Path=/;
                HttpOnly"
24
25     # Bước 3: Phân quyền
26     if not is_authenticated and not is_public:
27         return b"HTTP/1.1 401 Unauthorized\r\n..." # hoặc 302
                redirect
28
29     # Bước 4: Routing -> API handler
30     response = master_api_handler(req, resp)
31     return response

```

3.2 Quản lý Session

Thành phần	Mô tả
ACTIVE_SESSIONS	Dictionary RAM: session_id → username
db/sessions_id.txt	Lưu persistent: sid user expire_timestamp
db/account.txt	Tài khoản: username:password
SESSION_TTL	86400 giây (24 giờ)

Endpoint	Method	Mô tả
/api/login	POST	Xác thực, trả Set-Cookie: session_id=...
/api/logout	POST	Xóa session, đặt Max-Age=0
/api/me	GET	Trả tên user hiện tại

4 Xử lý HTTP Request và Response

4.1 Module Request (daemon/request.py)

```

1 def prepare(self, request, routes=None):
2     self.method, self.path, self.version = self.extract_request_line(
        request)

```

```

3     self.headers = self.prepare_headers(request)
4     self.body    = self.fetch_headers_body(request)[1]
5     # Parse Cookie
6     self.cookies = {}
7     for pair in cookie_str.split(';'):
8         if '=' in pair:
9             k, v = pair.strip().split('=', 1)
10            self.cookies[k] = v
11     self.auth = self.headers.get('authorization', None)
12     if routes:
13         self.hook = routes.get((self.method, self.path))

```

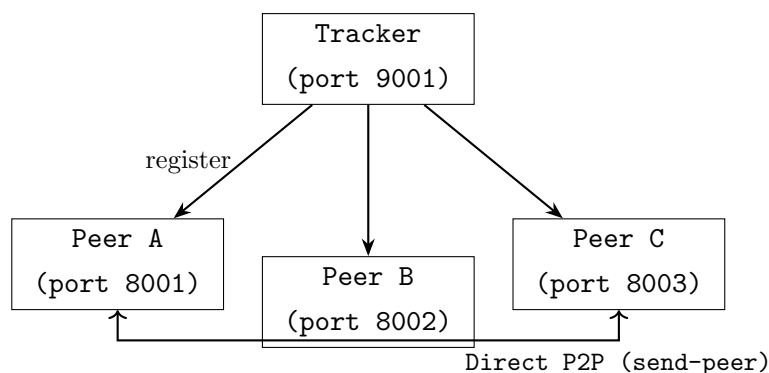
4.2 Module HttpAdapter (daemon/httpadapter.py)

Sau khi refactor, HttpAdapter có 3 method chính:

- `process_request(raw_msg, addr)`: Lỗi HTTP — dùng chung cho cả 3 mode.
- `handle_client(conn, addr, routes)`: Threading mode — `recv()` rồi gọi `process_request()`.
- `handle_client_coroutine(reader, writer)`: Async I/O — `await read()` rồi gọi `process_request()`.

5 Ứng dụng Chat Hybrid (Phần 2.3)

5.1 Kiến trúc Hybrid Client-Server + P2P



5.2 Giai đoạn khởi tạo (Client-Server)

Listing 4: trackerapp.py — đăng ký peer

```

1 @app.route('/submit-info', methods=['POST'])
2 def submit_info(req):

```



```

3     resolved_ip = resolve_peer_ip(ip, req.remote_addr)
4     PEER_REGISTRY[name] = {"ip": resolved_ip, "port": port, "last_seen": now}
5
6 @app.route('/get-list', methods=['POST'])
7 def get_list(req):
8     cleanup_online(now)    # Xóa peer offline (TTL 30s)
9     peers = [{"name": n, **info} for n, info in PEER_REGISTRY.items()
10              if ONLINE.get(n)]
11     return json_response({"code": 1, "peers": peers})

```

5.3 Giai đoạn Chat (Peer-to-Peer, không dùng asyncio)

ChatApp mới dùng **blocking socket** thay vì `asyncio.open_connection()`, và dùng `threading` thay vì `asyncio.gather()`:

Listing 5: `send_to_peer_sync()` — gửi P2P bằng blocking socket

```

1 def send_to_peer_sync(target_ip, target_port, payload):
2     body = json.dumps(payload).encode('utf-8')
3     http_request = (
4         f"POST /internal/receive-msg HTTP/1.1\r\n"
5         f"Content-Length: {len(body)}\r\nConnection: close\r\n\r\n"
6     ).encode() + body
7     s = socket.socket(AF_INET, SOCK_STREAM)
8     s.settimeout(5)
9     s.connect((target_ip, int(target_port)))
10    s.sendall(http_request)
11    s.recv(4096)    # Đọc và bỏ qua response
12    s.close()

```

Listing 6: `broadcast_peer()` — gửi đồng thời bằng threading

```

1 @app.route('/broadcast-peer', methods=['POST'])
2 def broadcast_peer(req):
3     payload = {"from": sender, "message": message, ...}
4     threads = []
5     for name, info in PEER_CACHE.items():
6         if name == sender_name: continue
7         t = threading.Thread(
8             target=send_to_peer_sync,
9             args=(info["ip"], info["port"], payload),
10            daemon=True

```

```

11         )
12         t.start(); threads.append(t)
13     for t in threads: t.join(timeout=5)

```

Listing 7: Nhan va lay tin nhan

```

1 @app.route('/internal/receive-msg', methods=['POST'])
2 def internal_receive_msg(req):
3     MESSAGE_QUEUE.append(json.loads(req.body))
4     return json_response({"code": 1, "message": "Received"})
5
6 @app.route('/poll-messages', methods=['POST'])
7 def poll_messages(req):
8     msgs = list(MESSAGE_QUEUE)
9     MESSAGE_QUEUE.clear()
10    return json_response({"code": 1, "messages": msgs})

```

6 Benchmark: Threading vs Callback

6.1 Phương pháp

Script `benchmark_th_cb.py` gửi N request đến 2 server, đo throughput và latency:

```

1 python benchmark_th_cb.py --th-port 9010 --cb-port 9020 -n 1000 --step
   50 --max-c 500

```

6.2 Kết quả đo lường

Concurrency	TH RPS	CB RPS	TH Avg(ms)	CB Avg(ms)	Winner
50	1010	1547	36.8	13.5	Callback
100	1049	1603	72.2	25.2	Callback
150	273	1556	148	37.3	Callback
200	344	343	200	136	Callback
300	405	501	251	234	Callback
400	416	339	314	333	Threading
500	443	419	388	437	Threading

6.3 Phân tích

- **Callback thắng** khi concurrency thấp-vừa (≤ 300): 1 thread không tốn overhead tạo thread mới, `select()` xử lý nhiều kết nối ngắn rất hiệu quả. Latency trung bình thấp

hơn 2–4 lần.

- **Threading thắng** khi concurrency rất cao (>400): Callback bị bottleneck vì 1 thread phải tuần tự xử lý từng callback. Thread pool cho phép song song thực sự trên multi-core.
- **Callback đặc biệt vượt trội** ở concurrency 150 (1556 vs 273 RPS): Do threading tạo quá nhiều thread đồng thời gây context switch overhead.

7 Cấu trúc file dự án

```

1 MMT_Assignment1/
2     daemon/
3         eventloop.py      # Custom select() Event Loop ( M I )
4         backend.py        # 3 mode: threading / callback
5         httpadapter.py    # process_request() dùng chung
6         request.py        # HTTP parser
7         response.py       # HTTP response builder
8         asynaprous.py     # Micro-framework routing (không
9         asyncio)
10        api.py            # API handlers
11    apps/
12        chatapp.py        # P2P chat (sync socket + threading)
13        trackerapp.py     # Peer registry
14    www/                  # HTML pages
15    db/                   # account.txt, sessions_id.txt
16    start_backend.py      # Entry: --mode threading/callback
17    start_chatapp.py      # Entry: chat peer
18    start_tracker.py      # Entry: tracker
19    benchmark_th_cb.py    # Benchmark threading vs callback
20    test_chatapp.py       # Test toàn bộ chức năng

```

8 Kết luận

Bài tập lớn đã thực hiện được các yêu cầu chính:

1. **Non-blocking HTTP Server tự xây**: Xây dựng EventLoop dựa trên `select.select()` trong `daemon/eventloop.py`, **không dùng asyncio hay bất kỳ framework bên ngoài nào**. Tuân thủ tinh thần “develop your own framework using the provided non-blocking mechanisms” của đề bài.
2. **Phân tầng rõ ràng**:

- Tầng I/O: `EventLoop` + `SelectHTTPServer` (select).
 - Tầng HTTP: `HttpAdapter.process_request()` (dùng chung 3 mode).
 - Tầng API: `master_api_handler()` + routes.
3. **Xác thực HTTP**: HTTP Basic Auth (RFC 7235) + Cookie session (RFC 6265), phân quyền public/private/API path.
 4. **Chat Hybrid**: Client–Server (Tracker) + P2P trực tiếp dùng blocking socket + threading cho broadcast đồng thời.
 5. **Benchmark**: Callback mode vượt trội ở concurrency thấp–vừa (tối đa $\times 5.7$ RPS so với threading ở $c=150$), Threading phù hợp hơn ở concurrency rất cao.
 6. **AsynapRous Framework**: Micro-framework decorator routing tự implement, không phụ thuộc thư viện ngoài.